TP 4 Web Service Soap

Ce TP à pour objectif de :

- Organisation des Travaux Pratiques (git et github)
- Comprendre l'architecture d'un Web Service SOAP
- Implémenter un Web Service SOAP avec **JAX-WS**
- Publier un service SOAP
- Générer et consommer un client SOAP
- Tester le service avec **SoapUI/Postman**

Partie 0. Organisation des Travaux Pratiques

Afin de garder un projet propre et bien organisé :

- Créer un répertoire principal nommé *`WebIntegration`*
- À l'intérieur, créer Un sous-dossier par TP

```
WebIntegration/
├── TP1/
├── TP2/
├── TP3/
```

Initialiser Git et Github

- Sur la ligne de commande, accéder au dossier du projet *WebIntegration*
- Vérifier l'installation de Git

```
git --version
```

- Configurer votre compte github

```
git config --global user.name "votre_user_name"
git config --global user.email "votre_user_email"
```

- vérifier la configuration

```
git config --list
```

- Initialiser le dépot git de votre projet

```
git init
```

- ajouter le dépôt distant github

```
git remote add origin
https://github.com/username/WebIntegration.git
```

- Premier commit et push

```
git add .
git commit -m "Initialisation du dépôt web integration Labs"
git branch -M main
git push -u origin main
```

- Aller sur GitHub et vérifier que le projet a bien été envoyé

A chaque nouveau TP

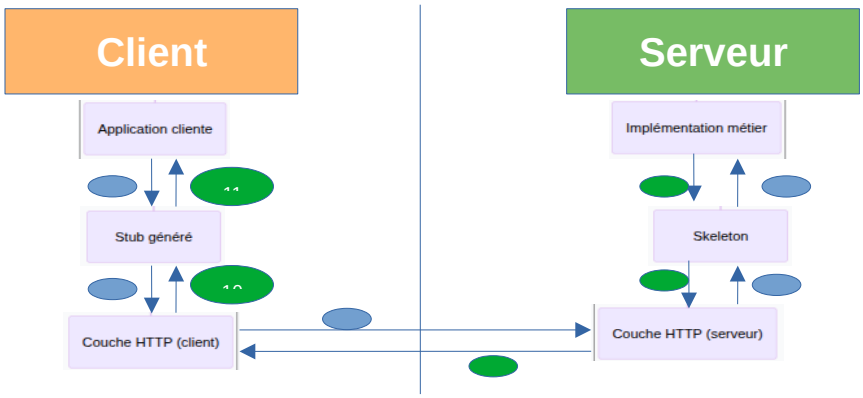
```
git init
git add TP2
git commit -m "Ajout du TP2 : …"
```

```
git push
```

I. Étude de cas : calculatrice à distance:

Dans un contexte de **systèmes distribués**, On souhaite mettre en œuvre une **calculatrice à distance** accessible via un **Web Service SOAP**, permettant à des clients distants d'effectuer des opérations de calcul sans implémenter la logique localement.

les applications clientes des applications Java, applications web, outils de test), indépendamment de la plateforme ou du langage utilisé, garantissant ainsi **interopérabilité**, **réutilisabilité** et **maintenabilité.**



Partie 1 – Création du Web Service SOAP

1. Quelles sont les étapes nécessaires pour la mise en place d'un service web soap ?
2. Le style adopté dans ce TP est RPC soap.
3. Créer un projet Maven pour le serveur. Le fichier pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <groupId>org.example</groupId>
   <artifactId>calculatrice-soap</artifactId>
   <version>1.0-SNAPSHOT</version>

   <properties>
      <maven.compiler.source>21</maven.compiler.source>
      <maven.compiler.target>21</maven.compiler.target>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
         <jaxws.version>2.3.5</jaxws.version>
   </properties>
```

```
    <!--dependencies -->
    <dependencies>
        <!-- JAX-WS pour SOAP -->
        <dependency>
            <groupId>com.sun.xml.ws</groupId>
            <artifactId>jaxws-rt</artifactId>
            <version>${jaxws.version}</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
            </plugin>
        </plugins>
    </build>
</project>
```

4. Créer l'interface *ICalculatrice* qui expose les méthodes accessibles via le web.

```
package org.example.ws;

import javax.jws.WebParam;
import javax.jws.WebService;
import javax.jws.WebMethod;
import java.util.List;

@WebService
public interface ICalculatrice {
    @WebMethod
    double add(@WebParam(name = "nbr1") double x,
               @WebParam(name = "nbr2") double y);
    @WebMethod
    double multiply(@WebParam(name = "nbr1") double x,
                    @WebParam(name = "nbr2")double y);
}
```

5. Ajouter les méthodes de soustraction et de division
    - Ajouter la gestion d'exception pour le cas de la division (ArithmeticException)
6. Créer la classe CalculatriceImp qui implémente le service SOAP.

```
package org.example.ws;
import javax.jws.WebService;

@WebService(endpointInterface = "org.example.ws.ICalculatrice")
public class CalculatriceImp implements ICalculatrice{
    @Override
    public double add(double x, double y) {
        System.out.println("addition de "+x+" et "+y);
        return x+y;
    }
    @Override
    public double multiply(double x, double y) {
        System.out.println("multiplication de " +x+ "et "+y);
        return x*y;
```

```
    }
}
```

7. compléter l'implémentation des autres méthodes
8. Publier le Web Service SOAP à l'aide du conteneur léger JAX-WS.

```
package org.example.ws;
import javax.xml.ws.Endpoint;

public class ServeurSOAP {
    public static void main(String[] args) {
        String url ="http://localhost:9000/calculatrice";
        System.out.println("demarrage du serveur...");
        Endpoint.publish(url, new CalculatriceImp());
        System.out.println("service soap démarré sur "+url);
        System.out.println("wsdl disponible sur "+url+"?wsdl");
    }
}
```

9. Démarrer le serveur soap
10. Accéder au WSDL et remplir le tableau suivant :

Élément WSDL	Nom trouvé	Rôle
`portType`		Contrat des opérations
`binding`		Protocole et encodage SOAP
`service`		URL de déploiement
`message`		Structure des paramètres
`types`		Schéma XML des objets

Partie 2 – Test du Web Service avec SoapUI

- Pour tester les méthodes disponibles, on utilisera l'application SoapUI.
- Installer l'application SoapUI.
- Créer un nouveau projet SOAP et préciser le nom du projet et le chemin du fichier WSDL publié par le serveur.
- Tester les différents opérations disponibles (addition, soustraction, etc.)

Partie 3 – Consommation via un client Java

1. Créer un nouveau projet maven calculatriceClient. Ajouter le contenu du fichier pom.xml :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>SoapClient</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
```

```
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.sun.xml.ws</groupId>
      <artifactId>jaxws-rt</artifactId>
      <version>2.3.5</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>com.sun.xml.ws</groupId>
        <artifactId>jaxws-maven-plugin</artifactId>
        <version>2.3.5</version>
        <executions>
          <execution>
            <goals>
              <goal>wsimport</goal>
            </goals>
            <configuration>
              <wsdlUrls>
                <wsdlUrl>http://localhost:9000/calculatrice?wsdl</wsdlUrl>
              </wsdlUrls>
              <packageName>com.example.client</packageName>
              <keep>true</keep>
            </configuration>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

2. Quel outil Java permet de générer automatiquement les classes client à partir du WSDL ?
3. Afin de créer les différents éléments nécessaires (le proxy SOAP, les types JAXB et classe du service) :

- **Méthode 1 :** utiliser la commande

```
wsimport -keep -s src/main/java http://localhost:9000/calculatrice?wsdl
```

ou bien

- **Méthode 2 :** exécuter la commande maven (maven doit être installer) au niveau de la racine du projet :

```
mvn clean generate-sources
```

4. Écrire le code du client Java pour consommer le Web Service Calculatrice.

```
package org.example;

import com.example.client.ICalculatrice;
import com.example.client.CalculatriceImpService;

public class Main {
  public static void main(String[] args) {
    System.out.println("Hello world!");
    CalculatriceImpService service = new CalculatriceImpService() ;
    ICalculatrice port = service.getCalculatriceImpServicePort();
    double sum = port.add(50,40);
```

```
        System.out.println("la somme est :"+sum);
    }
}
```

- Tester l'appel à d'autres méthodes.
- Ajouter un autre client de votre choix pour tester le web service SOAP (python, C#, etc.)

II Étude de cas : Banque à distance:

Dans le monde bancaire, les banques offrent à leurs clients la possibilité d'effectuer des opérations sur leurs comptes depuis **n'importe quel GAB (guichet automatique bancaire)**, même si le GAB appartient à une **autre banque**.

Pour permettre cette interopérabilité, on veut mettre en place un **Web Service SOAP centralisé** qui expose les opérations bancaires suivantes :

- Consultation du solde
- Retrait d'argent
- Dépôt
- récupérer un objet compte caractérisé par son id, solde et date de création (à ignorer dans la sérialisation java<→XML)

1. Créer un projet ou vous allez Créer le Web Service SOAP
2. Test du Web Service avec SoapUI et Postman
3. Créer un client java et un client (python par exemple) de votre choix.

III Étude de cas : magasin ElectroShop

L'entreprise "ElectroShop" , une chaîne de magasins d'électronique présente dans plusieurs villes. Chaque jour, les ventes sont enregistrées dans un système central. Les données collectées sont les suivantes :

Date de la vente, Magasin (identifiant, ville), Produit (nom, catégorie, prix unitaire), Quantité vendue, Montant total de la transaction

Pour garantir l'intégrité transactionnelle et la sécurité des échanges critiques (mises à jour de ventes et communication avec l'ERP), l'utilisation de SOAP s'impose ; le service exposera les opérations de base :

```
boolean ajouterVente(String magasin, String produit, int quantite, double prixUnitaire, Date dateVente)
double consulterCA(String magasin, Date dateDebut, Date dateFin)
List<String> topProduits(String magasin, Date dateDebut, Date dateFin, int n)
double predireVentes(String magasin, Date jour)
```

La prédiction est effectuée en appliquant une régression linéaire sur l'historique récent (5 derniers jours) pour détecter une tendance et projeter la valeur du jour suivant.